ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Angelina Panyananda**
- **Chinese Name: 鄭明美**
- **Email: angiepcnl@gmail.com**
- **Student ID: 115065434**
- **University: National Tsing Hua University**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. 594
2. Isolating a specific category keeps the analysis focused on the relevant domain and reduces noise from unrelated topics. For example, filtering for `sci.med` helps frequent pattern mining identify medical words that commonly occur together, rather than patterns from mixed topics. It also reduces the amount of data to process, improving efficiency.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://www.ibm.com/docs/it/SS3RA7_18.3.0/pdf/ModelerCRISPDM.pdf

---

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

Dropping these rows would discard 40% of the documents even though their text is still usable, losing valuable information. It could also introduce bias if some categories are more likely to have missing labels, leaving those categories underrepresented.

One alternative is unsupervised clustering: convert the text into numerical features, such as TF-IDF, and group similar documents using K-means. This uses the documents without requiring category labels. The resulting clusters are discovered groups, not necessarily the original categories.

So, we use the clusters to explore the documents, rather than automatically treating them as the original labels.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://scikit-learn.org/stable/modules/clustering

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No. The chart shows similar overall proportions, but not an exact match. X.sample(n=1000) randomly selects documents without enforcing a fixed number from each category, so some variation is expected.
2. A rare category may have too few documents—or none at all—in the sample. This can prevent the analysis or model from capturing patterns in that category. For example, if only 1% of documents belong to a category, a random sample of 100 documents could contain zero examples from it. Stratified sampling helps by sampling within each category to approximately preserve the original category proportions.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://scikit-learn.org/stable/modules/cross_validation.html#stratification

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. The number of features increases (from 35482 to 35788)
2. Sparsity generally decreases, the percentage of zeros becomes smaller. Common stop words occur in many documents, so their added columns contain relatively many nonzero counts. The matrix therefore becomes less sparse, although it can still contain mostly zeros.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

1. As the variance threshold increases from 0.0 to 0.05, the vocabulary decreases from 35,482 to 1,491 terms. A higher threshold removes more low-variance features.
2. Words that occur in very few documents with small counts, or have nearly identical counts across all documents, have variance close to zero and are eliminated first. Common words have low variance only when their counts are similar across documents.
3. Near-zero variance features usually provide little information for distinguishing documents, so removing them can reduce dimensionality, memory use, and computation with little effect on accuracy. However, this is not guaranteed: rare words may help identify minority categories, so model performance should be checked after filtering.

Threshold Number of terms kept (keep == True)

0.0 : 35482

0.01 : 5089

0.05 : 1491

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.VarianceThreshold

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

3. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
4. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
5. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. The results contain both general words and domain-specific terms. Pearson's top 10 include general words such as "thanks," "looking," and "hi," alongside technical terms such as "vga," "video," and "card." Spearman's top 10 show clearer topic-related vocabulary, including "graphics," "files," "image," "3d," and "program."
   Both tables are sorted by absolute correlation, so they also include negatively correlated words. For example, "people" has a negative correlation, meaning higher counts are associated with documents outside comp.graphics.
2. Pearson measures a linear relationship using actual word counts. Spearman measures a monotonic relationship using ranks, calculating Pearson correlation

on the ranked values. Thus, Spearman is less affected by the magnitude of extreme word counts.

3. A word can have a high Spearman rank correlation with a topic but a much lower Pearson correlation when it consistently appears more often in documents about that topic, yet a few documents repeat it an unusually large number of times. For example, "goal" might appear 0 times in each non-sports document but 2, 3, 4, and 1,000 times in four sports documents. Spearman is high because all sports documents have higher word counts than the non-sports documents. Pearson is lower because the extreme count of 1,000 creates a large difference among documents that share the same topic label.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

In these results, decreasing minSup from 9 to 3 increased both execution time and the number of discovered maximal patterns. Runtime increased from approximately 0.11 seconds to 2.64 seconds, while the pattern count increased from 710 to 9,547. The lower threshold allows less frequent word combinations to qualify, expanding the search and requiring more computation.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

k specifies the number of highest-support patterns to return, whereas minSup specifies the minimum support required for a pattern to qualify. My Top-K FAE runs returned 100 patterns with k = 100 and 500 patterns with k = 500. MaxFPGrowth does not request a fixed output count: it returns maximal patterns appearing in at least the specified number of documents.

Answer (3) — Maximal vs standard frequent patterns:

Standard frequent-pattern mining returns all word combinations meeting the support threshold, including their frequent subsets. Maximal frequent-pattern mining keeps only patterns that have no larger frequent superset. For example, if {doctor, patient, treatment} is frequent and cannot be extended, standard mining also returns its three individual words and three two-word combinations. Maximal mining represents these seven patterns with just the three-word pattern. This makes the output more concise, although it does not preserve the exact support counts of the omitted subsets.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://udaylab.github.io/PAMI/html/frequentPatternMaximalmaxFPGrowth.html

---

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. Each distinct word in X_counts represents one dimension, so each document has tens of thousands of coordinates. A standard scatter plot can display only two or three dimensions, making it impossible to visualize all the original dimensions directly. The matrix is mathematically valid, but showing it in a 2D or 3D plot requires reducing its dimensions.
2. We must first apply dimensionality reduction algorithms. The techniques listed in Section 5.5 are PCA, t-SNE, and UMAP. These methods transform each document into two or three coordinates for visualization while aiming to preserve useful information or relationships between documents. We can then color the points by category to examine how well the categories separate.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html

https://umap-learn.readthedocs.io/en/latest/

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. PCA compresses most documents into a small, heavily overlapping region, with a few distant outliers. t-SNE spreads the documents out more, but the category colors remain substantially mixed. UMAP shows clearer partial separation and some small isolated groups, although the four categories still overlap. Therefore, UMAP reveals more visible structure in these plots, but it does not perfectly separate all four categories.
2. PCA is a linear method that preserves directions with the greatest variance, without using category labels. In word-count data, this variation may reflect document length or extreme word counts rather than differences between topics. Reducing the data to only two or three principal components can therefore lose information useful for separating categories. t-SNE and UMAP preserve neighborhood relationships through non-linear mappings, which can reveal more complex groups of similar documents. However, they cannot guarantee category separation
3. t-SNE focuses on keeping similar documents close together locally, rather than preserving distances between separate clusters. Therefore, a larger gap between two clusters does not necessarily mean their topics are more different. For example, a cluster appearing twice as far away does not mean it is twice as dissimilar. The spacing can also change with the algorithm's settings.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

https://distill.pub/2016/misread-tsne/

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. Four binary columns are created, one for each category. Each document has a 1 in its category's column and 0 in the others.

Category	Binary vector
0	[1, 0, 0, 0]
1	[0, 1, 0, 0]
2	[0, 0, 1, 0]
3	[0, 0, 0, 1]

2. When these integers are treated as numerical input features, they introduce a false order and distance between categories. For example, KNN would treat category 0 as closer to category 1 than to category 3. Linear regression would treat each increase in the category number as a meaningful numerical step. However, these numbers are only category identifiers.
3. One-hot binarization gives each category its own column, removing the artificial numerical ordering. With all four columns, every pair of different categories has

the same Euclidean distance. Thus, no category is considered closer to another simply because of its assigned number.

AI used for this answer: Yes

If yes, how: concept explanation and grammar

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html

https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features

---